Universidade Federal de Santa Maria
Prof. Cesar Tadeu Pozzer
Data: 04/12/2024

P2 – Computação Gráfica Avançada

Utilize figuras e equações sempre que possível.

1. Em relação aos algoritmos de Dijkstra e A*, comente sobre a fórmula f' = g + h'. Em qual situação os dois algoritmos são iguais. Comente sobre a influência dos valores de g e h' no caminho encontrado. Dê duas situações que não se pode utilizar o A*, justificando.
2. Um NPC de um jogo do tipo FPS (Ex: Quake) é um agente, um programa ou um simples objeto de software? Justifique e faça comentários em relação às discussões e exemplos apresentados em aula sobre Programas X Agentes X Objetos.
3. Para um jogo com portais (pontos de tele transporte instantâneo para outros locais do cenário), comente quais são os algoritmos de busca mais apropriados e quais não são apropriados.
4. Explique o que é retornado de um steering behavior qualquer e como esse valor é aplicado na movimentação do personagem. Utilize figuras e fórmulas.
5. Descreva um processo de movimentação para desvio de obstáculos estáticos com uso de waypoints. O agente deve possuir um vetor direção e um ângulo de campo de visão. Utilize figuras e equações.

Gabarito resumido (sem figuras):

1. O custo é dado pela soma dos nós já processados + um valor heurístico. h **não** pode ser maior que a distância real. Se g = 1, a busca fica em largura. Se h = 0, A* e dijkstra ficam iguais. Não pode usar A* em grafos negativos, grafos com portais (pois a heurística pode ser maior que a real) e grafos que não tenham coordenadas espaciais.
2. É um agente pois observa o ambiente, envia e recebe mensagens, toma decisões, tem autonomia, realiza ações que modificam o ambiente, o que leva **ele a ter novas percepções no futuro, e que por sua vez fazem ele tonar decisões diferentes das anteriores**, pois o mundo ao seu redor foi modificado.
3. Jogos com portais são jogos em espaços. Logo, existem distâncias, ou seja, grafos com peso. Logo, não pode usar nada que não trate peso (largura, profundidade). Não da para usar A* pois a heurística não vai funcionar no portal, pois a distância estimada vai ser maior que a real. Então sobre o Dijkstra ou outro algoritmo que trate pesos.
4. Só olhar no material. Retorna uma força. Com a força obtém-se a aceleração, que é então aplicada para mudar a velocidade, que por sua vez muda a posição final.

```
steeringForce = steering->calculateSteering(this, targetPos, steeringType);
Vector2 acceleration = steeringForce / massa;
vel += (acceleration * time_elapsed);
vel.truncate(maxVel);
pos += vel * time_elapsed;
```

5. Somente usar um steering behavior wall avoidance não vai funcionar. Deve-se definir waypoints de contorno do obstáculo, associados a cada obstáculo. Quando detectado uma colisão, dentro do campo de visão, o algoritmo escolhe o novo waypoint do obstáculo que pode estar mais na direção do alvo ou mais próximo (pode desviar para esquerda ou direita). Faz um seek para esse waypoint. Após desviar o obstáculo, segue a lista de waypoits já definido ou segue usando o algoritmo A*, dijkstra, etc.